

Name: Suchir Jolapara

BU ID: U08469805

Application Package Name: com.example.mobilitylens

Environment and SDK Configuration: For this assignment, I ran and tested the application using a Medium Phone emulator running API Level 28 (Android 9.0, Pie). When configuring the project, I selected the following SDK parameters:

- **minSdk (28):** This defines the oldest version of Android that the application is permitted to run on. I chose API 28 (Pie) as a middle-ground floor that covers most active Android devices while keeping compatibility overhead low for a first Compose project.
- **targetSdk (37):** This specifies the API version the application was explicitly tested against. Setting this tells the Android operating system which security models, permissions, and behavioral changes the app expects and complies with, at the most current platform level available.
- **compileSdk (37):** This instructs Android Studio on which SDK version to use when compiling the source code, granting access to the latest available APIs while writing the app, even though it's designed to still run on older devices back to minSdk 28.

Project Architecture: The application relies on several core configuration and source files, each serving a distinct purpose:

- **MainActivity.kt** (located in app/src/main/java/com/example/mobilitylens/): This serves as the primary entry point of the app. It is where the Android OS instantiates the interface, hosting the Compose UI tree and managing the state variables that dictate what the user sees.
- **AndroidManifest.xml** (located in app/src/main/): This essential configuration file provides the Android system with critical metadata about the application. It declares the app's structural components, defines the application theme, and points to MainActivity as the launch point.
- **app-level build.gradle.kts** (located in gradle/): This file manages build instructions specific to the application module. It is where the SDK versions are defined and where external library dependencies (like Material 3 and Compose tooling) are implemented.
- **libs.versions.toml** (located in gradle/): This version catalog acts as a centralized hub to organize, declare, and manage dependency and plugin versions across the entire project, keeping the gradle files clean.
- **strings.xml** (located in app/src/main/res/values/): This file externalizes human-readable text strings from the Kotlin source code. Keeping text here simplifies copy edits and allows the application to be easily translated (localized) into other languages by swapping out resource files.

Rotation Observation: During testing, I navigated to a later dimension, typed a proposed feature into the text field, and rotated the emulator to landscape mode. Upon rotation, the application completely reset: the text field was wiped clean, and the screen reverted to the first dimension.

This total state loss occurred because the Android OS intentionally destroys and recreates the Activity upon a configuration change such as a screen rotation to reload resources optimized for the new layout dimensions. Because I used 'remember' for the state variables, the data only survived standard UI recompositions, not a full Activity lifecycle tear-down. Mobile applications must proactively preserve this information (using tools like rememberSaveable or ViewModels) because users naturally shift how they hold their devices; losing active input during a simple rotation results in a highly frustrating user experience.

Mobile-Design Decision: When designing the layout, I deliberately applied the fillMaxWidth() modifier to the text input field, while maintaining generous outer padding around all interactive elements. While a desktop application might cluster small inputs together because a mouse provides pixel-perfect precision, mobile users navigate using their thumbs. Stretching the interactive element across the screen creates a large, forgiving touch target. This makes the application comfortable to operate with one hand and prevents accidental misclicks, directly addressing the physical and ergonomic constraints highlighted in the "Input and Interaction" dimension.

AI Disclosure: I used Claude (Anthropic) throughout this assignment for guidance on setting up the Android Studio project, connecting it to GitHub, and understanding the Compose framework. Claude recommended an initial file structure, outlined the workflow, and guided my Kotlin syntax. I manually typed the code, rewrote the six dimension descriptions entirely in my own words to meet the assignment requirements, and resolved several bugs introduced during transcription. Additionally, I relied on the AI to troubleshoot errors, explain core Compose concepts like state and theming, and cross-reference my report's technical details with my project files. Finally, I independently ran and tested the application on an emulator to verify that all functionality operated correctly.